The Baltor Divergence Engine: 9-Stage Poly-Modal Ingestion, Three-Layer Epistemic Separation (Evidence, Interpretation, Exploration), and Controlled Model-Based A/B Evaluation

Taylor Amarel · Principal Investigator (Baltor.ai & DueCare) · Live Endpoint: https://divergence.baltor.ai · Code: https://github.com/Amarel-Taylor-Scott/polymodal-5w1h-ideation-graph-engine

Abstract & Summary: Proving that structured 3-layer epistemic separation, perspective simulation, and recursive 5W1H Protagonist/Antagonist interrogation outperform conventional single-pass LLM prompting on live gemma4:31b runs.

1. WHY CONVENTIONAL RAG AND PROMPTING CONVERGE ON THE WRONG ANSWER

Every piece of information should become the starting point for thousands of testable investigations rather than the endpoint of a single summarization or embedding call. When an unassisted language model is asked to summarize a multi-document packet or answer open-ended *Who / What / Where / When / Why* questions, it exhibits two predictable failures:

- **Surface-Claim Capture:** It repeats the explicit wording of the documents as if the surface framing were the underlying reality.
- **Epistemic Layer Conflation:** When asked to brainstorm or explore counterfactuals without strict layer boundaries, a system can generate thousands of variations of an unsupported claim and mistake their repeated appearance for independent evidence.

The Core Architectural Invariant — Three Strictly Separated Epistemic Layers:
1. **EVIDENCE Layer:** What the immutable original source actually says (`SHA-256` provenance, page/image-region coordinates, and exact cross-record contradictions).
2. **INTERPRETATION Layer:** What a model, statutory rule, or analytical process infers from that evidence (explicit Protagonist vs. Antagonist hypotheses with calibrated `[confidence, strength]` scores).
3. **EXPLORATION Layer:** What becomes possible when we change assumptions, project through historical/thinker perspectives, ask contrastive `NOT` questions, or synthesize new executable verifier rules.
These layers reference each other by typed edges, but they are never silently merged.

2. THE NINE INTERCONNECTED ENGINES OF THE BALTOR DIVERGENCE PIPELINE

Engine Stage	Primary Responsibility	Principal Typed Output
1. Universal Ingestion	Accept documents, PDFs, images, SQLite/CSV databases, chat logs, and code	Immutable source records + `SHA-256` root
2. Recursive Partitioning	Construct overlapping structural, semantic, page/figure, and stochastic views	Hierarchical chunk DAG + coordinates
3. Mutation & Remix	Strip fluff, shift register, supplement via RAG, and transcode modalities	Lineage-tagged derived variants
4. Perspective Simulation	Analyze material through thinker lenses & historical constraints (e.g., Apollo 13)	Alternative interpretations + 10x challenges
5. Recursive Interrogation	Pursue direct, contrastive (`NOT`), residual, and Protagonist/Antagonist 5W1H	`[confidence, strength]` scored arrays
6. Knowledge Graph	Separate lexical co-occurrence and model suggestions from verified causal edges	3-Layer traversable attribution graph
7. Divergence Optimization	Measure DPP diversity volume, detect convergence, and redirect exploration	De-conformed exploration branches
8. Scenario & Capability Discovery	Formulate failure scenarios, counterfactuals, and reusable harness tools	6-file Baltor component candidates
9. Validation & Learning	Execute `verifier.py` , retain empirical lift, and feed signals upstream	Verified components & re-entry triggers

3. CONTROLLED LIVE MODEL EXPERIMENT ON OLLAMA CLOUD (`GEMMA4:31B`)

To test whether the Baltor Divergence Engine genuinely outperforms a conventional LLM using the **exact same model weights** (`gemma4:31b` at `T = 0.3`), we executed paired live runs across two multi-hop investigative packets on October 17–18, 2026:

Domain & Experimental Arm (`gemma4:31b`)	Observed Latency	Prompt / Eval Tokens	Statutory & Causal Recall	Observed Analytical Behavior